

Q.1) Print the type of file and inode number where file name accepted through Command Line

[10 Marks]

```
#include <stdio.h>

#include <stdlib.h>

#include <sys/stat.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <filename>\n", argv[0]);
        return 1;
    }

    struct stat st;
    if (stat(argv[1], &st) == -1) {
        perror("stat");
        return 1;
    }

    printf("File: %s\n", argv[1]);
    printf("Inode Number: %ld\n", st.st_ino);

    printf("File Type: ");
    if (S_ISREG(st.st_mode)) printf("Regular File\n");
    else if (S_ISDIR(st.st_mode)) printf("Directory\n");
    else if (S_ISCHR(st.st_mode)) printf("Character Device\n");
    else if (S_ISBLK(st.st_mode)) printf("Block Device\n");
    else if (S_ISFIFO(st.st_mode)) printf("FIFO/Pipe\n");
    else if (S_ISLNK(st.st_mode)) printf("Symbolic Link\n");
    else if (S_ISSOCK(st.st_mode)) printf("Socket\n");
    else printf("Unknown\n");

    return 0;
}
```

Q.2) Write a C program which creates a child process to run linux/ unix command or any user defined program. The parent process set the signal handler for death of child signal and Alarm signal. If a child process does not complete its execution in 5 second then parent process kills child process

[20 Marks]

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
```

```
pid_t child_pid;
```

```
void sigchld_handler(int sig) {
    int status;
    pid_t pid = waitpid(child_pid, &status, WNOHANG);
    if (pid > 0) {
        printf("Child process terminated.\n");
        if (WIFEXITED(status))
            printf("Exit status: %d\n", WEXITSTATUS(status));
        exit(0);
    }
}
```

```
void sigalrm_handler(int sig) {
    printf("Child did not finish in 5 seconds. Killing child...\n");
    kill(child_pid, SIGKILL);
}
```

```
int main() {
    signal(SIGCHLD, sigchld_handler);
    signal(SIGALRM, sigalrm_handler);

    child_pid = fork();
```

```
if (child_pid < 0) {
    perror("fork");
    exit(1);
}

if (child_pid == 0) {
    // Child process: execute a command, e.g., sleep 10
    execlp("sleep", "sleep", "10", NULL);
    perror("execlp");
    exit(1);
} else {
    // Parent process: set alarm
    alarm(5); // 5-second timer
    // Wait for child
    while(1) {
        pause(); // Wait for signals
    }
}

return 0;
}
```